

MODUL 4
PENGEMBANGAN APLIKASI MOBILE

State Hoisting, Perubahan Konfigurasi, dan Side-effect

Dosen Pengampu: Muhammad Aminul Akbar, S.Kom., M.T.



Oleh

Yosefina Agustine K. Ofong
NIM. 245150400111030

PROGRAM STUDI SISTEM INFORMASI
DEPARTEMEN SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

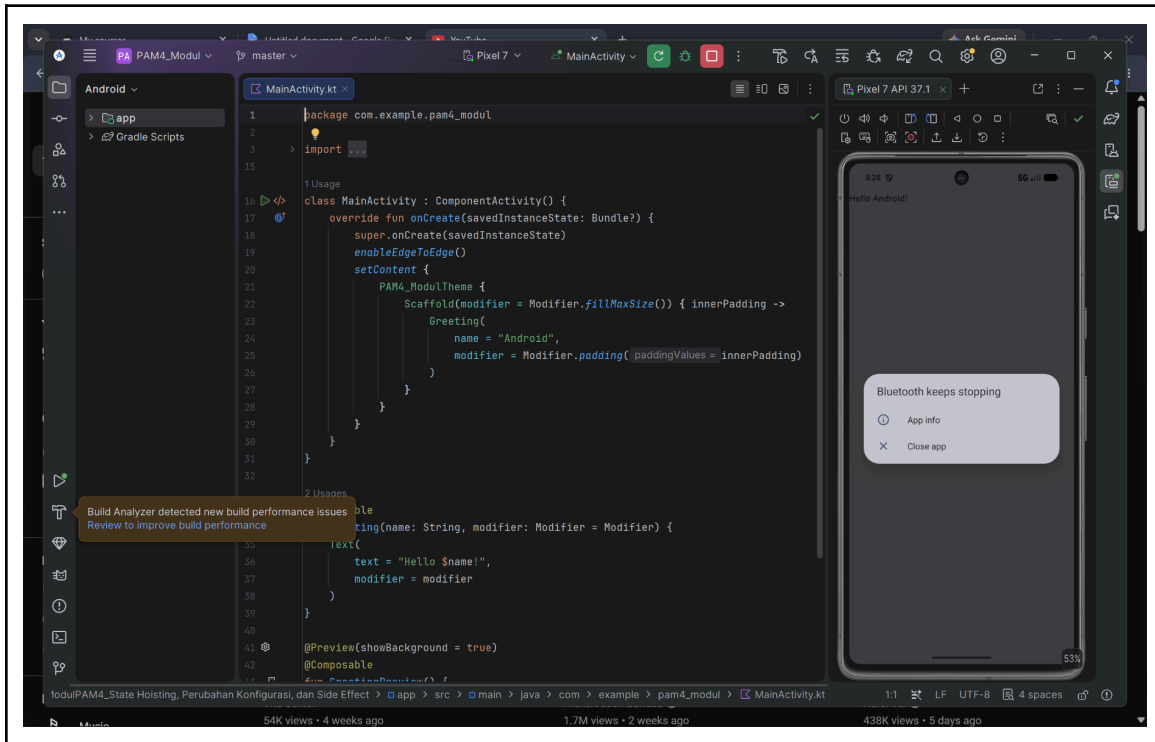
MALANG

2026

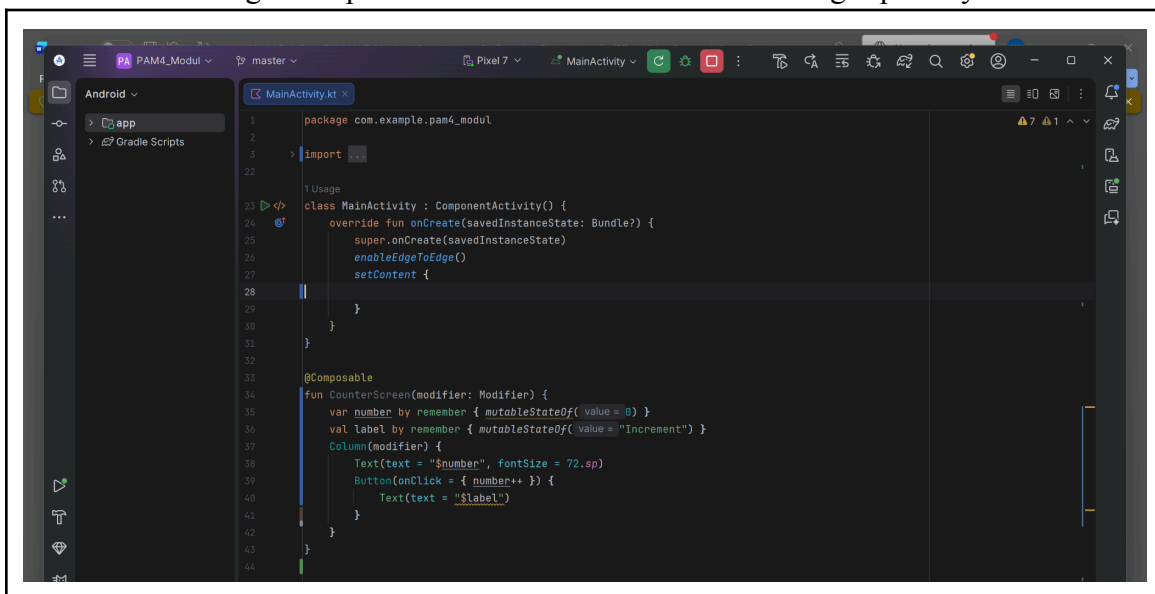
4.5.1 State Hoisting

Pra-Percobaan

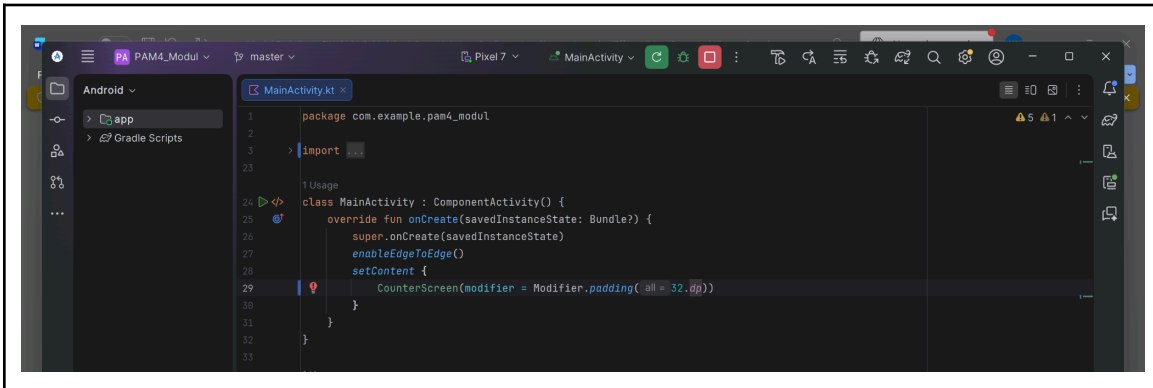
1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.



2. Menambahkan fungsi composable sesuai arahan modul dan mengimpor keyword

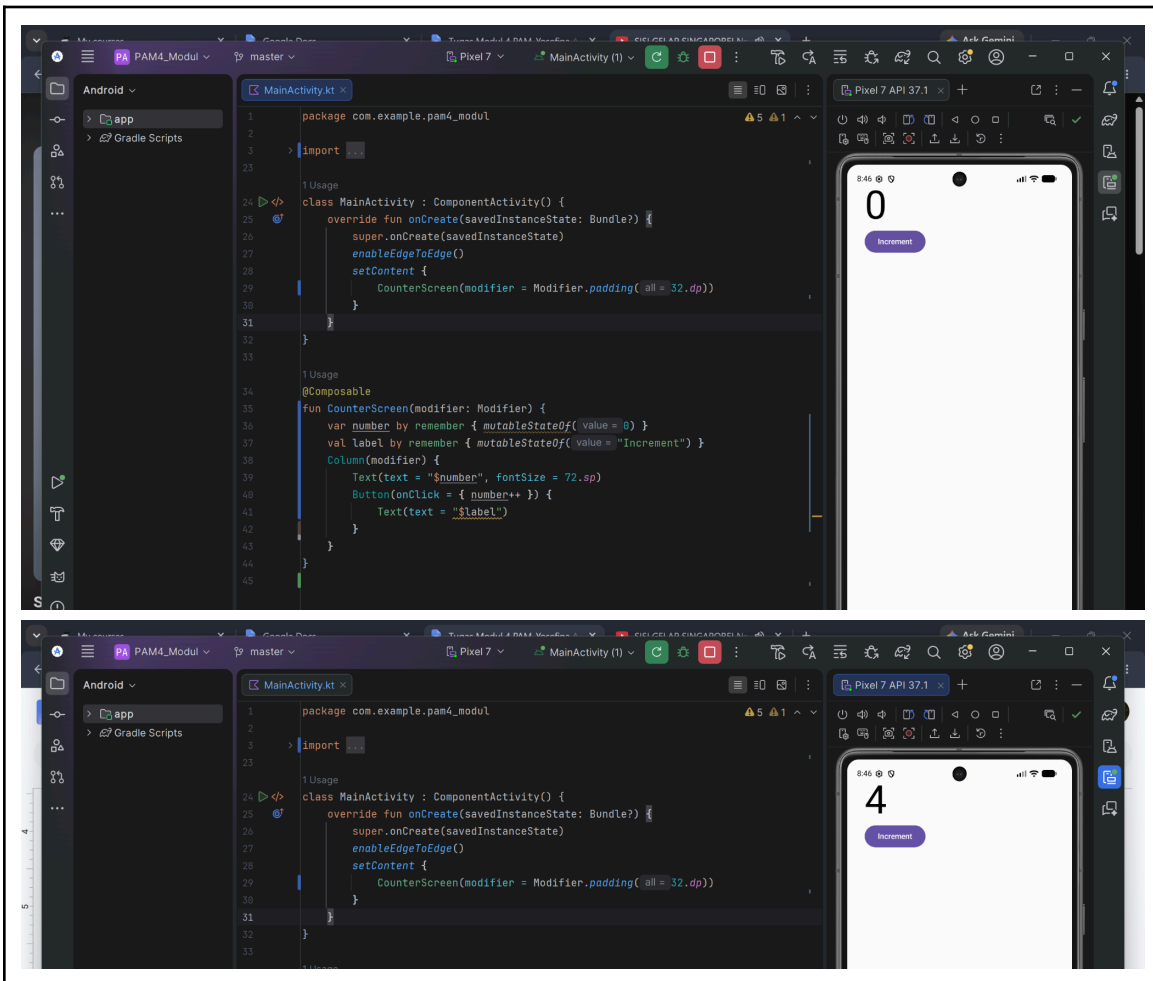


3. Ubah onCreate yang sudah ada dengan fungsi baru



Aktivitas Percobaan

1. Menjalankan program



- a. Apa yang terjadi setiap kali tombol “increment” di-klik beberapa kali?

Setiap kali tombol “Increment” diklik, angka yang ditampilkan bertambah satu. Nilai awalnya adalah 0, kemudian berubah menjadi 1, 2, 3, dan seterusnya sesuai jumlah klik.

- b. Apa yang menyebabkan hal tersebut terjadi?

Tombol menjalankan perintah `number++` pada callback `onClick`. Perintah tersebut menambah nilai `number` sebanyak satu. Karena `number` merupakan state yang dibuat menggunakan `mutableStateOf`, perubahan nilainya memicu recomposition pada bagian antarmuka yang membacanya sehingga angka pada layar diperbarui.

- c. Variabel *state* apa yang berperan pada `Composable Text` yang menampilkan perubahan angka pada `Text` tersebut?

Variabel state yang berperan adalah `number`. Variabel ini menyimpan angka yang ditampilkan oleh `Composable Text` dan memiliki nilai awal 0. Ketika nilainya berubah, `Text` menampilkan nilai `number` yang terbaru.

- d. Variabel state apa yang berperan untuk menampilkan label teks pada `Composable Button` yang bertuliskan “increment” tersebut?

Variabel state yang berperan adalah `label`, yang memiliki nilai awal “Increment”. Nilai label ditampilkan melalui `Composable Text` di dalam `Button` sehingga menjadi tulisan pada tombol.

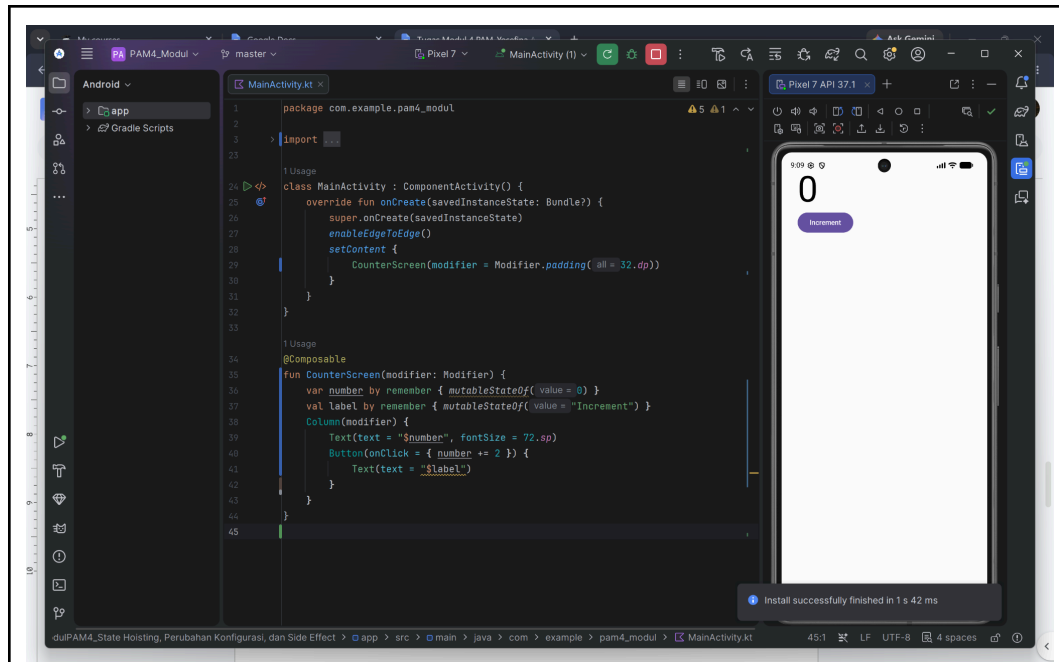
2. Mengubah sehingga saat tombol “increment” di-klik itu angka bertambah 2 kali.

- a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

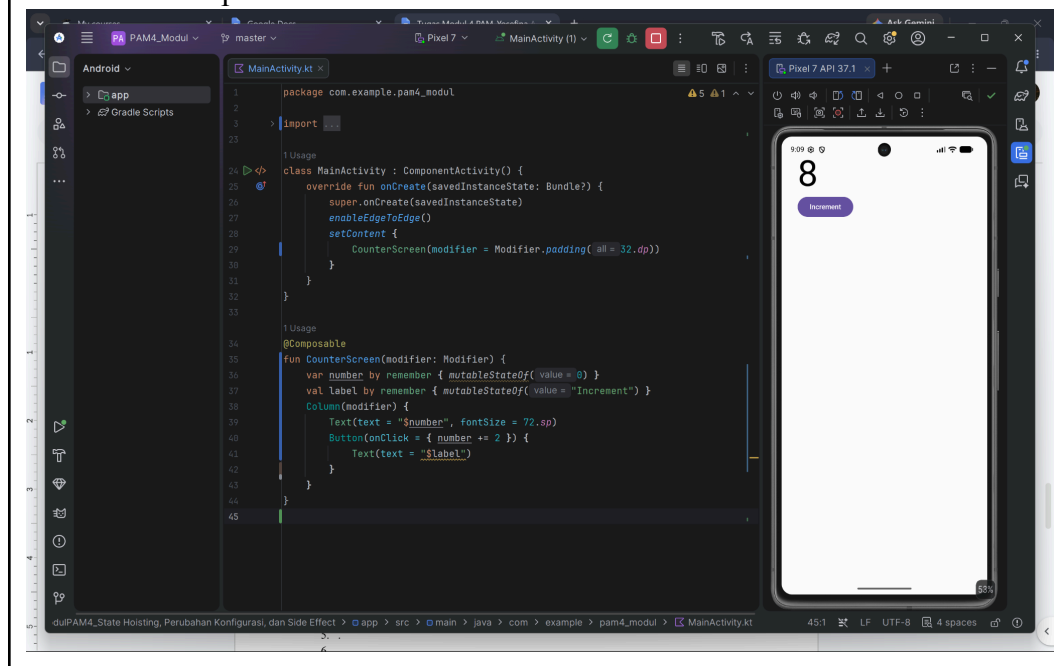
Perubahan dilakukan di dalam fungsi **`Composable CounterScreen`**, yaitu **mengganti perintah `number++` pada event `onClick` menjadi `number += 2`** agar setiap klik menambah angka sebanyak dua. Selain itu, nilai awal variabel label diubah dari “Increment” menjadi “Increment 2” agar tulisan tombol sesuai dengan fungsi barunya.

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

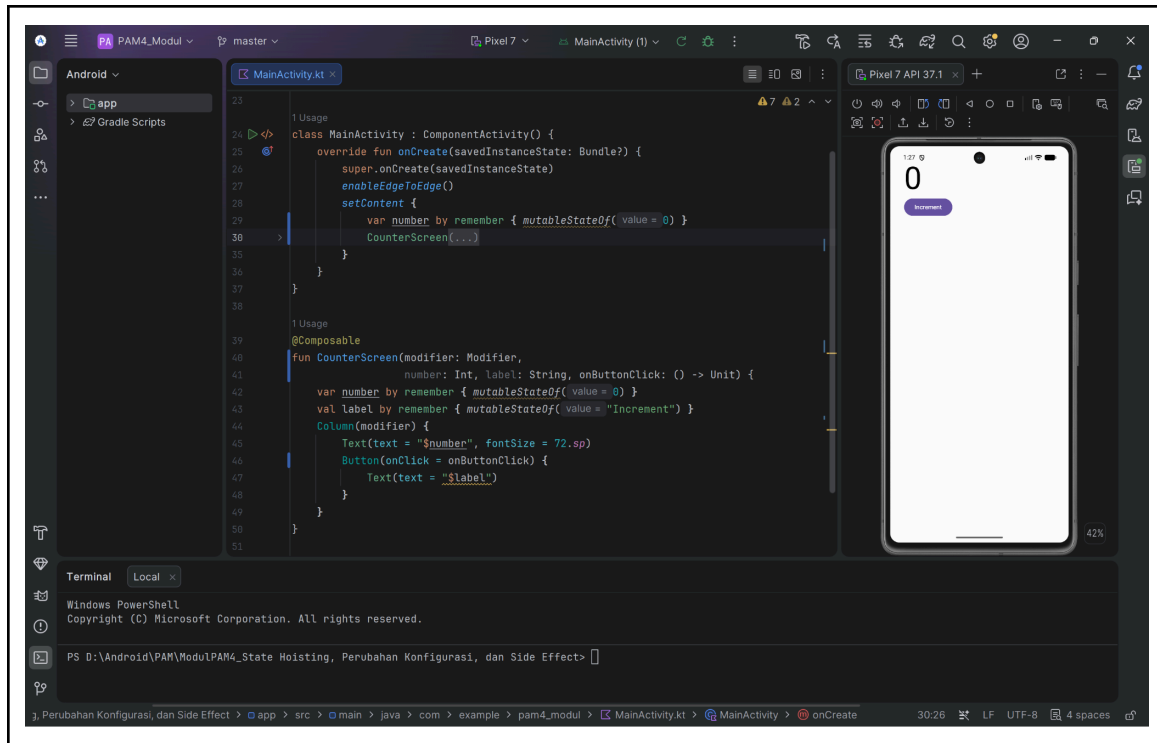
//sebelum diklik



//setelah beberapa kali diklik



3. Mengubah Signature code program sesuai perintah modul.
4. Menjalankan program



- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Setelah state hoisting diterapkan dengan benar, program dapat berjalan dengan baik. Tampilan dan perilakunya tetap sama apabila nilai awal, label tombol, dan operasi increment yang digunakan tidak berubah. Perbedaannya hanya pada pengelolaan state dan event yang dipindahkan dari CounterScreen ke composible induk. Namun, jika sebelumnya menggunakan `Increment 2` lalu diubah menjadi `number++`, pertambahan angka berubah dari dua menjadi satu.

- b. Di bagian kode program manakah saat itu proses increment dilakukan?

Proses increment dilakukan pada lambda `onClick` dalam pemanggilan CounterScreen, yang berada di dalam blok `setContent` pada fungsi `onCreate()`. Kode `onClick = { number++ }` menambah nilai `number` sebesar satu setiap kali tombol diklik.

- c. Dibagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State `number` untuk menampilkan angka pada Text dideklarasikan di dalam blok `setContent` pada fungsi `onCreate()`, sebelum pemanggilan CounterScreen,

melalui var number by remember { mutableStateOf(0) }. Sementara itu, label tombol diberikan melalui parameter label = "Increment". Dengan demikian, CounterScreen menerima nilai angka dan label dari induk tanpa mendeklarasikan state tersebut secara lokal.

5. Mengubah kode program sehingga:

- Label tombol "Increment" menjadi "Double"
- Nilai awal variabel number dari 0 menjadi 1.
- Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: number++ menjadi menggandakan nilai variabel number.
- Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

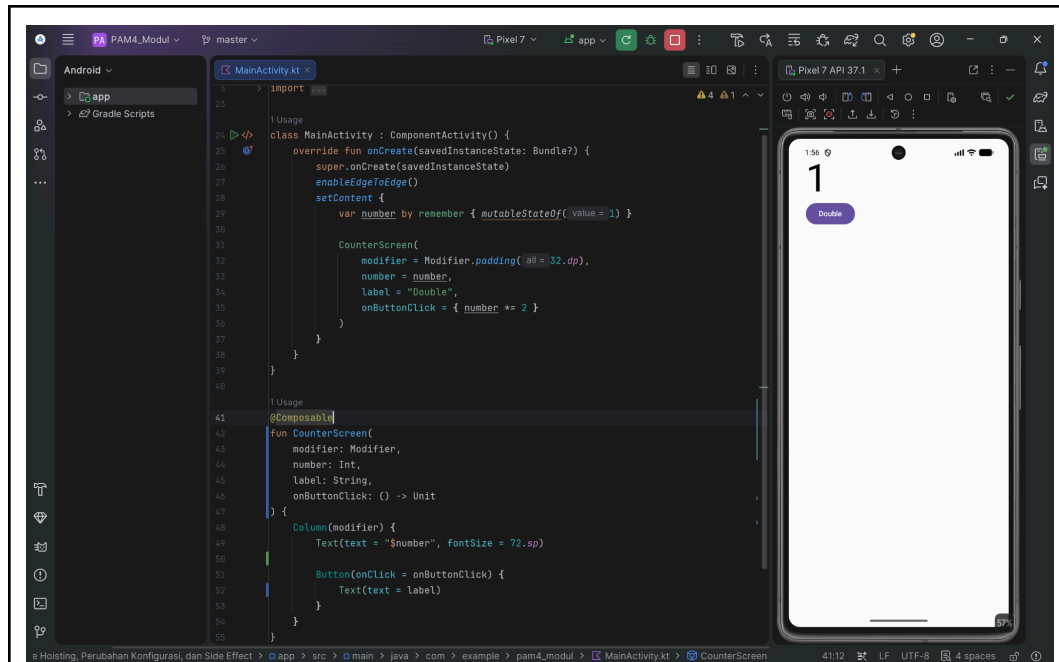
Kode program yang diubah:

```
1 Usage
24 <> class MainActivity : ComponentActivity() {
25     override fun onCreate(savedInstanceState: Bundle?) {
26         super.onCreate(savedInstanceState)
27         enableEdgeToEdge()
28         setContent {
29             var number by remember { mutableStateOf( value = 1) }
30
31             CounterScreen(
32                 modifier = Modifier.padding( all = 32.dp),
33                 number = number,
34                 label = "Double",
35                 onClick = { number *= 2 }
36             )
37         }
38     }
39 }
```

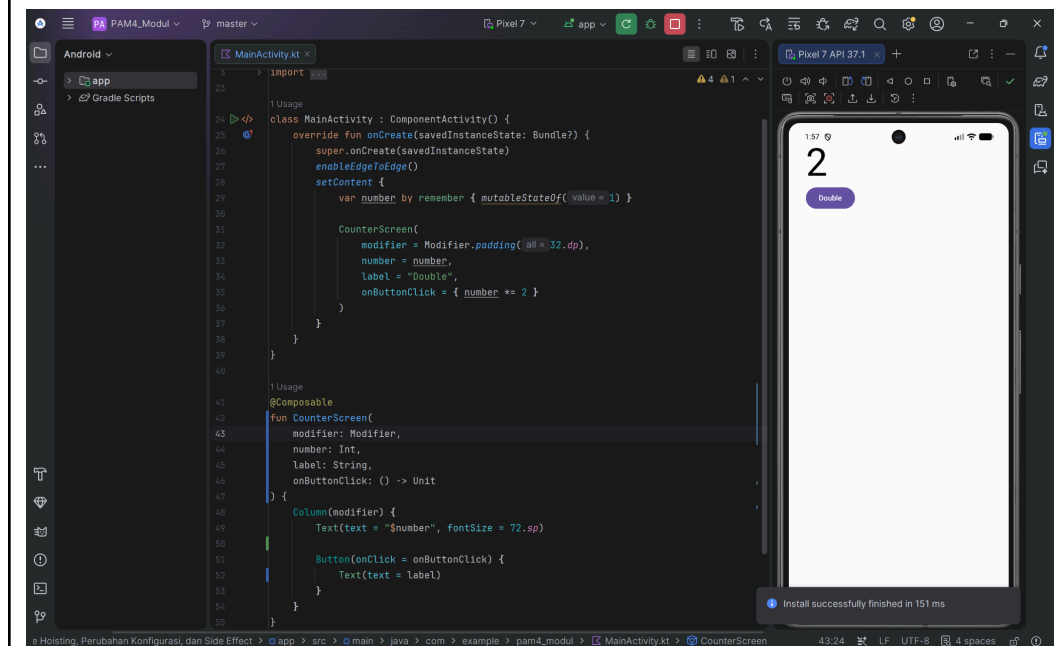
6. Menjalankan program

- Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

//Sebelum



//Sesudah



Penjelasan: nilai awalnya adalah 1, lalu saat tombol ‘increment’ di-klik value berubah menjadi 2, karena perkalian antara angka 1 dan 2.

- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Perubahan dilakukan di dalam blok setContent pada fungsi onCreate(), tepatnya

pada nilai awal state number serta argumen label dan lambda onClick saat memanggil CounterScreen.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada perubahan di dalam CounterScreen apabila state hoisting pada tahap sebelumnya sudah diterapkan dengan benar. Fungsi tersebut tetap menerima parameter number, label, dan onClick. Perubahan dari Increment menjadi Double cukup dilakukan pada nilai dan callback yang dikirimkan oleh induk.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

State hoisting memudahkan perubahan kebutuhan fungsional karena pengelolaan state dan logika tindakan dilakukan pada composable induk. Pada percobaan ini, operasi penambahan dapat diubah menjadi penggandaan tanpa mengubah fungsi CounterScreen. Dengan demikian, komponen tampilan lebih mudah digunakan kembali, dipelihara, dan disesuaikan dengan kebutuhan aplikasi.

4.5.2 Perubahan Konfigurasi

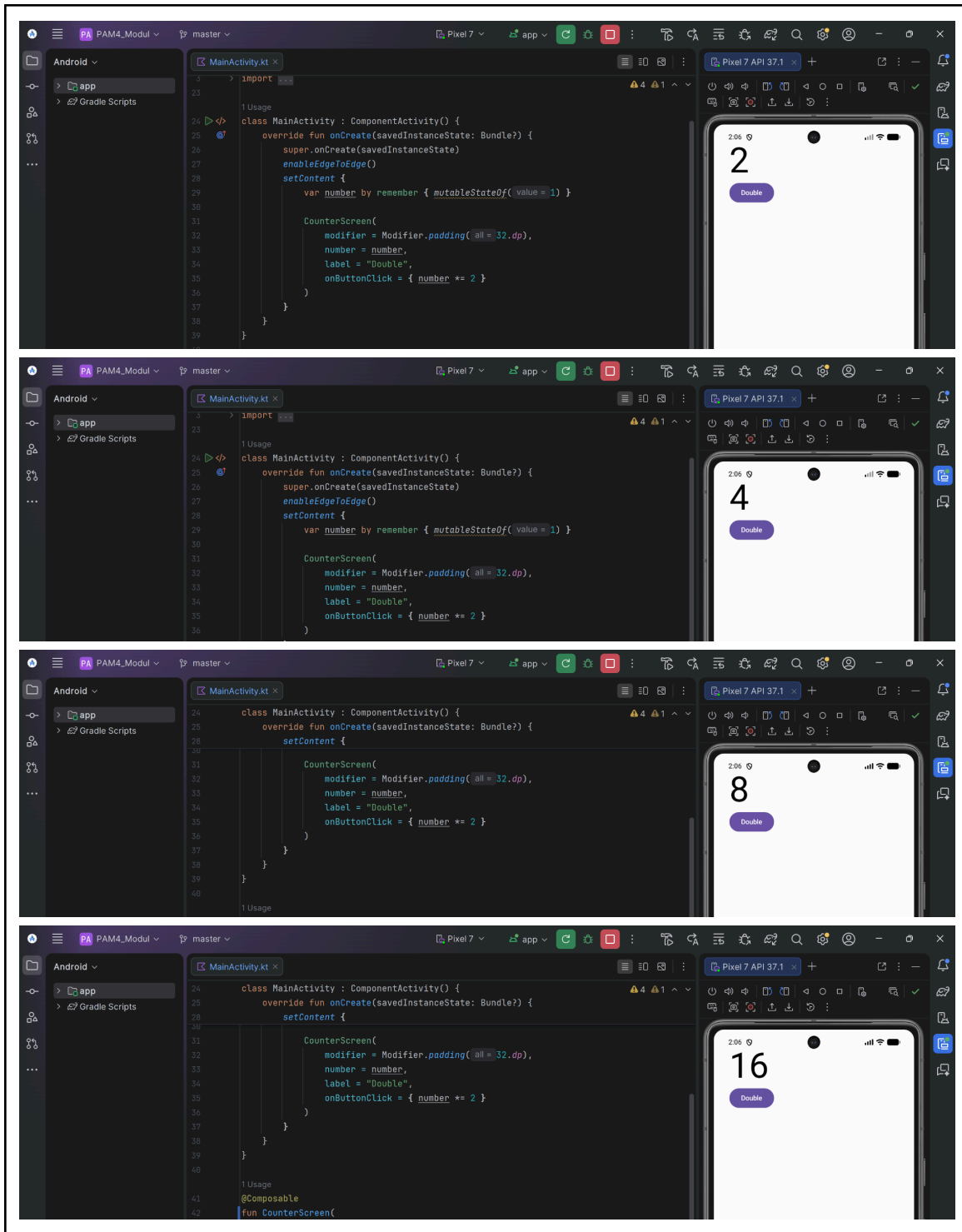
Pra-Percobaan

1. Tetap memakai project android studio yang sama
2. Program masih dapat berjalan seperti percobaan sebelumnya

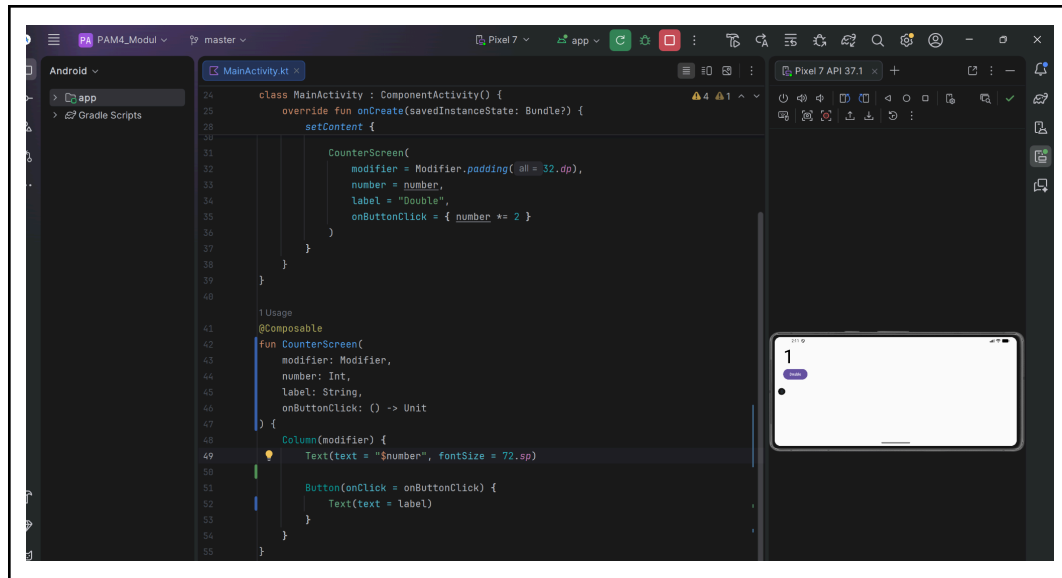
Aktivitas percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Angka yang muncul setelah tombol di-klik $2 \rightarrow 4 \rightarrow 8 \rightarrow 16$



2. Lakukan perubahan dari portrait ke landscape
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

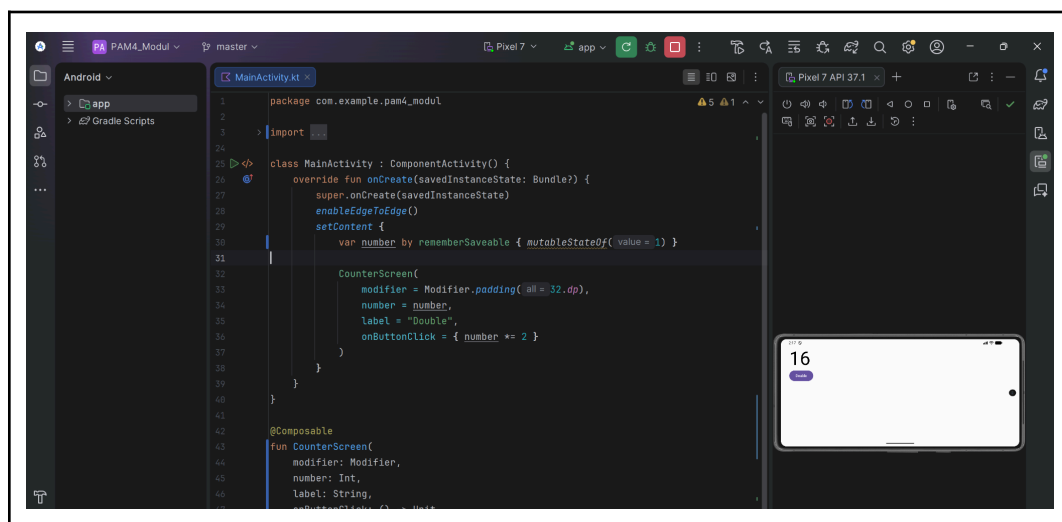


- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang ditampilkan kembali menjadi 1. Hal ini dikarenakan, State yang disimpan menggunakan remember tidak bertahan ketika Activity dibuat ulang, sehingga number kembali menggunakan nilai awal, yaitu 1. remember hanya mempertahankan nilai selama recomposition dalam composition yang sama.

3. Lakukan perubahan dari portrait ke landscape lagi.

- a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Angka yang tertampil 16, sama dengan nilai sebelum layar diputar. Hal ini terjadi karena `rememberSaveable` menyimpan nilai state melalui mekanisme saved instance state Android. Ketika Activity dibuat ulang akibat perubahan orientasi, nilai number dipulihkan sehingga tetap **8**, bukan kembali ke nilai awal 1.

- c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

`rememberSaveable` mempertahankan state selama recomposition sekaligus memungkinkan pemulihannya ketika Activity dibuat ulang, misalnya akibat rotasi layar. Sementara itu, `remember` hanya mempertahankan state selama recomposition dan kehilangan nilainya ketika Activity beserta composition-nya dibuat ulang. Data yang disimpan menggunakan `rememberSaveable` harus berupa tipe yang dapat disimpan atau menggunakan Saver yang sesuai.

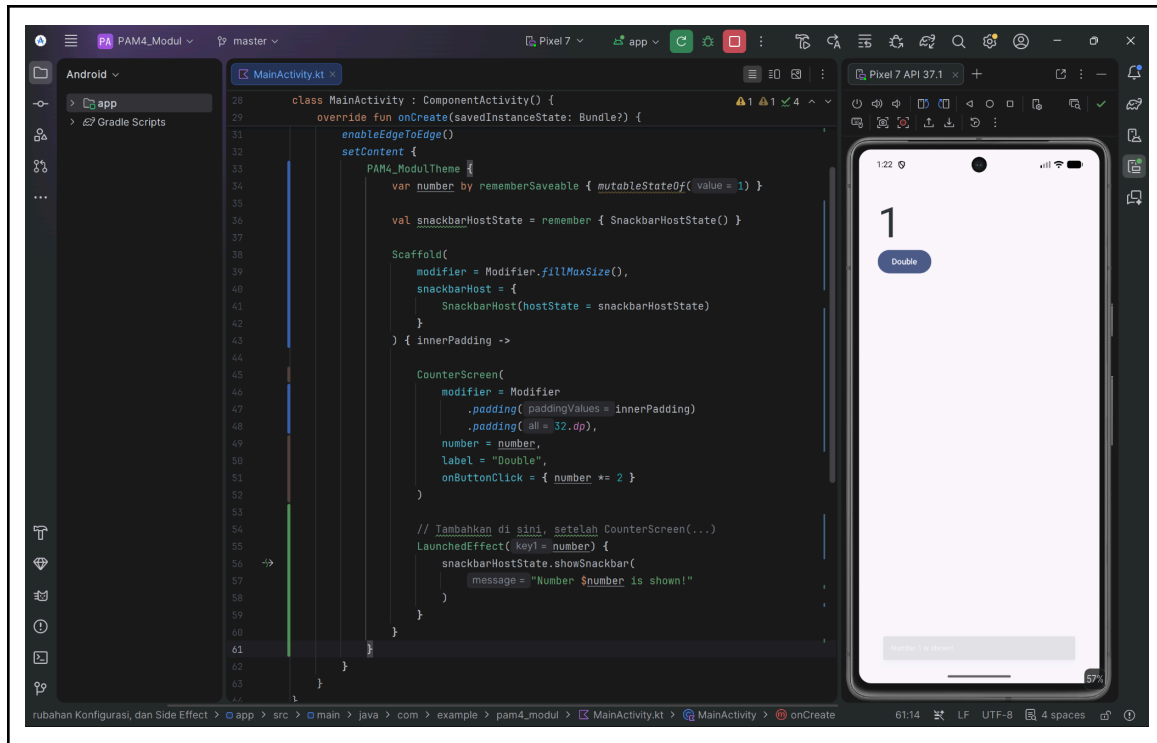
4. Beri contoh kasus nyata lain dimana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contohnya adalah menyimpan input nama pada formulir pendaftaran. Ketika pengguna sudah mengetik nama lalu memutar perangkat, `rememberSaveable` mempertahankan isi input tersebut sehingga pengguna tidak perlu mengetik ulang. Penerapannya mencegah hilangnya isian sementara akibat perubahan konfigurasi dan membuat penggunaan aplikasi lebih nyaman.

4.5.3 Launched Effect

Pra-Percobaan 1

1. Menggunakan folder android studio yang sama dengan percobaan sebelumnya.
2. Aplikasi dapat berjalan dengan baik.
3. Mengaitkan composable `SnackBar` di dalam `Scaffold`.
4. Menerapkan fitur Side-Effect dengan `Launched Effect`.

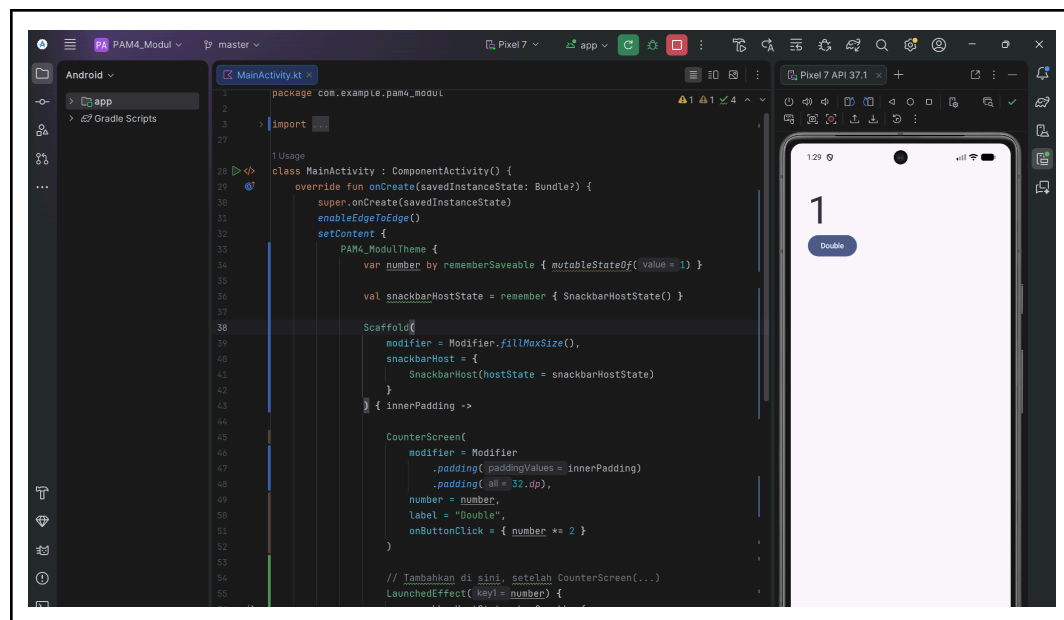


5. Import Dependency. Pemeriksaan jika semua kode dilakukan di dalam fungsi onCreate().

Aktivitas Percobaan

1. Jalankan Program

- Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?



- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Pesan ditampilkan karena `LaunchedEffect` otomatis dijalankan ketika pertama kali masuk ke `composition`. Oleh karena itu, kode untuk menampilkan `Snackbar` langsung dijalankan tanpa menunggu pengguna menekan tombol.

- c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

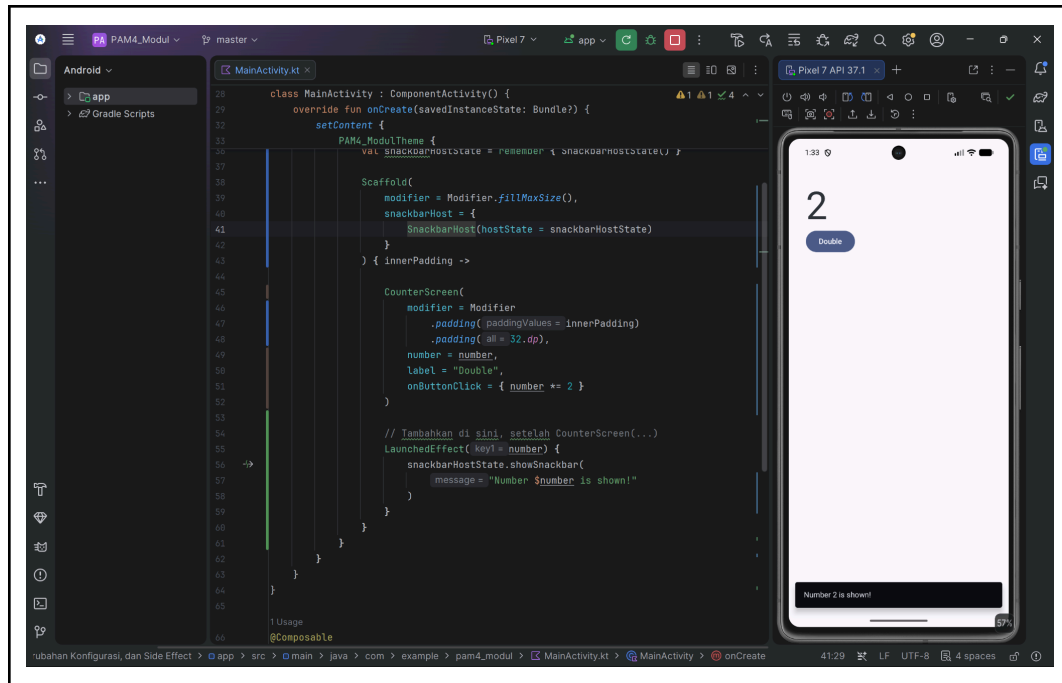
Pemicu awalnya adalah `LaunchedEffect(number)` yang masuk ke `composition`. Di dalamnya, fungsi `showSnackBar()` dipanggil untuk menampilkan pesan berdasarkan nilai `number`.

```
LaunchedEffect( key1 = number) {  
    snackbarHostState.showSnackBar(  
        message = "Number $number is shown!"  
    )  
}
```

2. Klik button “Double” satu kali

- a. Apa yang terjadi?

Angka berubah dari 1 menjadi 2, kemudian `Snackbar` menampilkan pesan “**Number 2 is shown!**”.

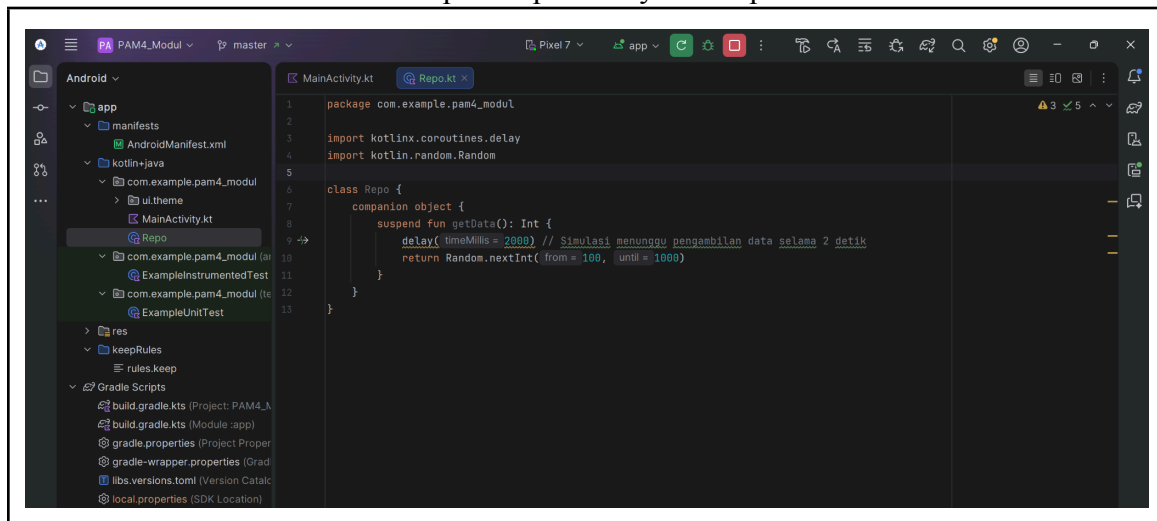


b. Mengapa hal tersebut terjadi? jelaskan!

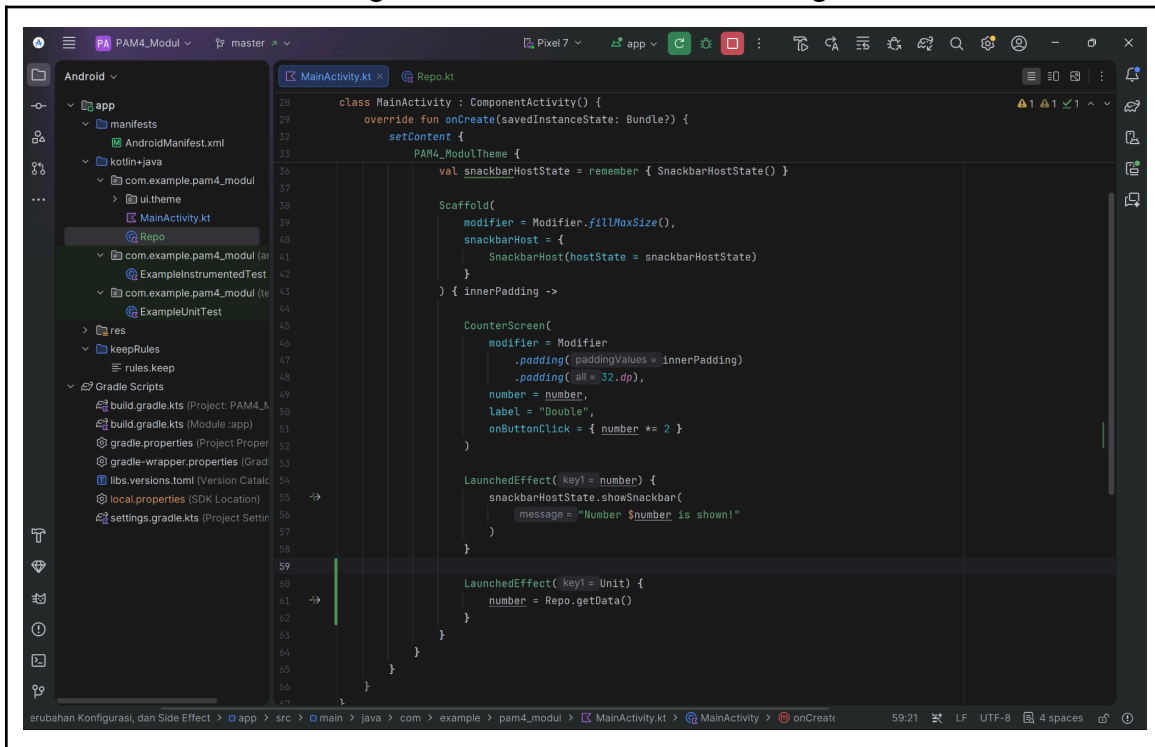
Tombol Double menjalankan callback `onClick = { number *= 2 }`, sehingga nilai number digandakan dari 1 menjadi 2. Perubahan state tersebut memicu pembaruan tampilan angka. Karena number juga digunakan sebagai key pada `LaunchedEffect`, efek dijalankan ulang dan menampilkan Snackbar dengan nilai terbaru.

Pra-Percobaan 2

3. Melanjutkan project dengan kode yang sama dari percobaan sebelumnya.
4. Membuat class kotlin baru dan import dependency sesuai perintah modul



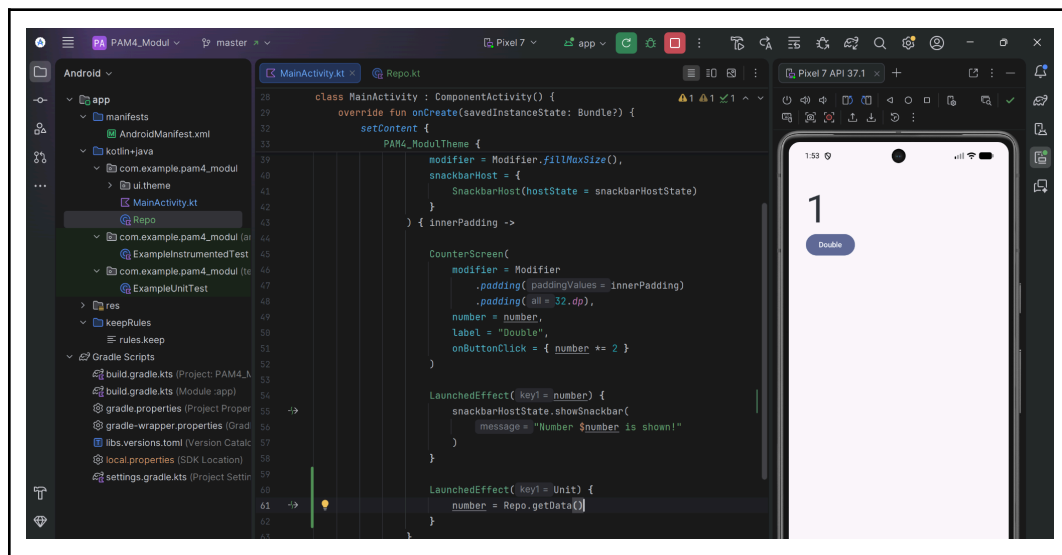
5. Menambahkan deklarasi fungsi `launchEffect` baru di dalam fungsi `onCreate`.



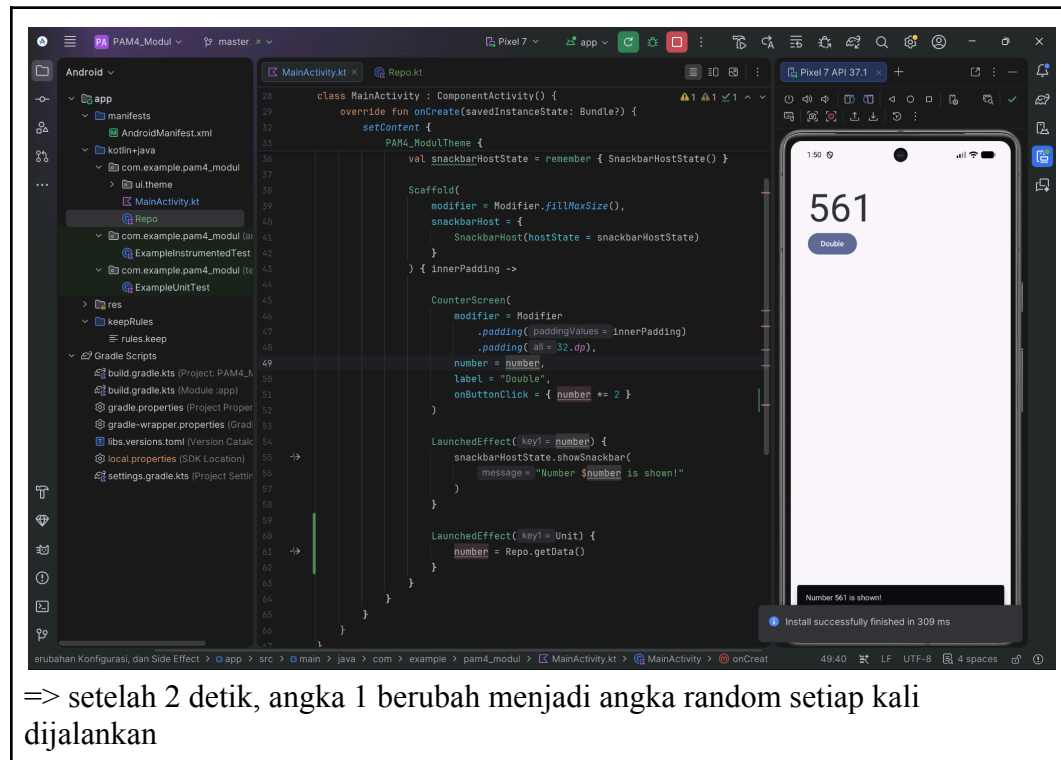
6. Memastikan semua sudah sesuai dengan arahan modul.

Aktivitas Percobaan

1. Jalankan program
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?



- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?



- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Perubahan angka dipicu oleh `LaunchedEffect(Unit)` yang memanggil `Repo.getData()` dan menyimpan hasilnya ke state `number`:

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

Di dalam `Repo.getData()`, `delay(2000)` mensimulasikan waktu tunggu dua detik, kemudian `Random.nextInt(100, 1000)` menghasilkan angka acak. Perubahan state `number` memperbarui angka pada layar sekaligus memicu `LaunchedEffect(number)` untuk menampilkan pesan terbaru:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar("Number $number is shown!")  
}
```

2. Jelaskan

- a. Apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

→ `LaunchedEffect(state)` menggunakan nilai `state` sebagai **key**, sehingga efek dijalankan ulang ketika nilai `key` tersebut berubah. Contohnya, `LaunchedEffect(number)` memperbarui pesan `Snackbar` setiap kali `number` berubah.

Sementara itu, `LaunchedEffect(Unit)` menggunakan `key` konstan yang tidak berubah. Efek tidak dijalankan ulang hanya karena terjadi `recomposition` atau perubahan `number`. Pada percobaan ini, efek digunakan untuk memulai pengambilan data ketika masuk ke `composition`.

- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

→ **Jawab:**

Bentuk	Waktu Dijalankan
<code>LaunchedEffect(state)</code>	Saat pertama kali masuk ke <code>composition</code> dan setiap kali nilai <code>key state</code> berubah.
<code>LaunchedEffect(Unit)</code>	Saat pertama kali masuk ke <code>composition</code> ; tidak diulang selama tetap berada di <code>composition</code> yang sama.

Ketika `key` berubah, `LaunchedEffect(state)` membatalkan `coroutine` sebelumnya jika masih berjalan, lalu menjalankan `coroutine` baru. Keduanya dibatalkan ketika keluar dari `composition` dan dapat dijalankan kembali jika masuk ke `composition` baru, misalnya setelah `Activity` dibuat ulang akibat rotasi layar.